

CCF-GESP C++二级

编程能力等级认证



第十课

多层循环 2

01 多层循环的应用



真题解析

选择题

(2023年9月) 以下C++代码实现**从大到小**的顺序输出N的所有因子，例如，输入N=18时，输出18 9 6 3 2 1，横线处应该填入()。

```
int N=0;  
cin>>N;  
for(_____)//此处填写代码  
    if(!(N%i))  
        cout<<i<<' ';
```

- A. ; ;
- B. int i=1; i<N; i++
- C. int i=N; i>0; i--
- D. int i=N; i>1; i--



真题解析

题目解析

```
int N=0;  
cin>>N;  
for(int i=N; i>0; i--) //此处填写代码  
    if(!(N%i))  
        cout<<i<<' ';
```

整数a除以整数b(b≠0)的**商**正好是整数而**没有余数**，b就是a的**因子**。

输入N=18时输出18 9 6 3 2 1

i:	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1
	✓									✓			✓			✓	✓	✓

```
for( int i=18; i>0; i-- ){  
    //找出18的因子  
}
```



真题解析

选择题

(2023年9月) 某班级人数不知, 连续输入成绩直到**输入负数停止**, 输入结束后求出平均成绩。在以下C++代码横线处应填入是(A)。

```
double totalScore=0; //总分
int studCount=0; //总人数
while(____){ //此处填写代码
    cin>>score;
    if(score<0)
        break;
    totalScore+=score;
    studCount+=1;
}
cout<<"平均分="<<totalScore/studCount;
```

A. true

B. false

C. True

D. False



真题解析

选择题

(2023年9月) 下面C++代码执行后的输出是(B)。

```
int cnt=0;
for(int i=1; i<13; i+=3)
    for(int j=1; j<i; j+=2)
        if(i*j%2==0)
            break;
        else
            cnt+=1;
cout<<cnt;
```

A. 1

B. 3

C. 15

D. 没有输出



真题解析

题目解析

```
int cnt=0;
for(int i=1; i<13; i+=3)
    for(int j=1; j<i; j+=2)
        if(i*j%2==0)
            break;
        else
            cnt+=1;
cout<<cnt;
```

i	j	$i*j\%2==0$
1		
4	1	当j=1, 结果为true, 执行break
7	1、3、5	当j=1, 结果为false, 执行cnt+=1 当j=3, 结果为false, 执行cnt+=1 当j=5, 结果为false, 执行cnt+=1
10	1	当j=1, 结果为true, 执行break



真题解析

选择题

(2023年9月) 下面C++代码用于判断 N 是否为质数（素数），约定输入 N 为**大于等于2**的正整数，请在横线处填入合适的代码(A)。

```
int N=0, i=0;
cout<<"请输入一个大于等于2的正整数: ";
cin>>N;
for(i=2; i<N; i++)
    if(N%i==0){
        cout<<"非质数";
        _____;//此处填写代码
    }
if(i==N)
    cout<<"是质数";
```

- A. break
- B. continue
- C. exit
- D. return



真题解析

(2023年3月) 百鸡问题🐔

【问题描述】

“百鸡问题”是出自我国古代《张丘建算经》的著名数学问题。大意为：“每只公鸡 5 元，每只母鸡 3 元，每 3 只小鸡 1 元；现在有 100 元，买了 100 只鸡，共有多少种方案？”

小明很喜欢这个故事，他决定对这个问题进行扩展，并使用编程解决：如果每只公鸡 x 元，每只母鸡 y 元，每 z 只小鸡 1 元；

现在有 n 元，买了 m 只鸡，共有多少种方案？

【输入描述】

输入一行，包含五个整数，分别为问题描述中的 x 、 y 、 z 、 n 、 m 。

约定 $1 \leq x, y, z \leq 10$, $1 \leq n, m \leq 1000$ 。

【输出描述】

输出一行，包含一个整数 C ，表示有 C 种方案。



真题解析

(2023年3月) 百鸡问题🐔

【输入样例1】

5 3 3 100 100

【输出样例1】

4

【样例1解释】

这就是问题描述中的“百鸡问题”。

4 种方案分别为：公鸡 0 只、母鸡 25 只、小鸡 75 只；

公鸡 4 只、母鸡 18 只、小鸡 78 只；

公鸡 8 只、母鸡 11 只、小鸡 81 只；

公鸡 12 只、母鸡 4 只、小鸡 84 只。

【输入样例2】

1 1 1 100 100

【输出样例2】

5151



真题解析

思路分析

列出题目给出信息：

- 鸡的价格：公鸡： x /只；母鸡： y /只；小鸡： z 只1元
- 手中钱数： n
- 购鸡总数： m

根据题目给出信息列出解题的条件：

- 公鸡数 $\times x$ + 母鸡数 $\times y$ + 小鸡数 $\div z = n$
- 公鸡数 + 母鸡数 + 小鸡数 = m



真题解析

思路分析

① 输入公鸡价格 x ，母鸡价格 y ， z 只小鸡一元，总钱数 n ，购买鸡数 m 。

```
int x, y, z, n, m;
```

```
cin >> x >> y >> z >> n >> m;
```



真题解析

思路分析

- 公鸡数 $\times x$ + 母鸡数 $\times y$ + 小鸡数 $\div z = n$
- 公鸡数 + 母鸡数 + 小鸡数 = m

② 使用枚举法列出所有公鸡、母鸡可能的数量、计算小鸡数量。

1) 列举公鸡数 a 的可能性

0	1	2	3	4	
---	---	---	---	---	-------

➡ $a \times x \leq n$ 并且 $a \leq m$

2) 列举母鸡数 b 的可能性

0	1	2	3	4	
---	---	---	---	---	-------

➡ $a \times x + b \times y \leq n$ 并且 $a + b \leq m$

3) 根据已知条件计算小鸡数 c ➡ (总钱数-买公鸡母鸡钱数) $\div z$

```
for(int a=0;  $a \times x \leq n \&\& a \leq m$ ; a++){ ➡ 枚举公鸡数量 (从0开始)
    for(int b=0;  $a \times x + b \times y \leq n \&\& a + b \leq m$ ; b++){ ➡ 枚举母鸡数量 (从0开始)
        int c=(n-a*x-b*y)*z; ➡ 计算小鸡数量
    }
}
```



真题解析

思路分析

- 公鸡数 $\times x$ + 母鸡数 $\times y$ + 小鸡数 $\div z = n$
- 公鸡数 + 母鸡数 + 小鸡数 = m

③ 统计符合条件的购买方案数量。

记录方案数量

```
int cnt=0;
for(int a=0; a*x<=n&&a<=m; a++){
    for(int b=0; a*x+b*y<=n&&a+b<=m; b++){
        int c=(n-a*x-b*y)*z;
        if(a+b+c==m) cnt++;    数+1
        公鸡数 + 母鸡数 + 小鸡数 = m
    }
}
```



真题解析

【完整代码】

```
#include <iostream>
using namespace std;
int main(){
    int x,y,z,n,m;
    cin>>x>>y>>z>>n>>m;
    int cnt=0;
    for(int a=0; a*x<=n&&a<=m; a++){
        for(int b=0; a*x+b*y<=n&&a+b<=m; b++){
            int c=(n-a*x-b*y)*z;
            if(a+b+c==m) cnt++;
        }
    }
    cout<<cnt<<endl;
    return 0;
}
```



真题解析

(2023年6月) 找素数

【问题描述】

小明刚刚学习了素数的概念：如果一个大于 1 的正整数，除了 1 和它自身外，**不能被其他正整数整除**，则这个正整数是素数。现在，小明想找到两个正整数A和B之间（包括A和B）有多少个素数？

【输入描述】

输入 2 行，第一行包含正整数A，第二行包含正整数B。约定 $2 \leq A \leq B \leq 1000$ 。

【输出描述】

输出一行，包含一个整数C，表示找到C个素数。

【输入样例1】

2

10

2	3	4	5	6	7	8	9	10
✓	✓		✓		✓			

【输出样例2】

4

【输入样例1】

98

100

98	99	100
----	----	-----

【输出样例2】

0



真题解析

思路分析

- ① 输入正整数a, 正整数b。

```
int a = 0, b = 0;  
cin >> a >> b;
```

- ② 创建变量cnt, 统计素数个数。

```
int cnt = 0;
```

- ③ 统计正整数 a 到 b 范围内的素数个数 (包含a和b) 。

```
for (int n = a; n <= b; n++){  
    //判断当前正整数n是否为素数  
    //累加素数的个数  
}
```



真题解析

判断一个正整数是否为素数

素数：如果一个大于 1 的正整数，除了 1 和它自身外，**不能**被其他正整数**整除**，则这个正整数是素数。

例：判断9是否为素数

2	3	4	5	6	7	8
---	---	---	---	---	---	---

↓
 $9 \% 3 \rightarrow 0$ **非素数**

```
//判断n是否为素数
bool flag = true;
for (int i = 2; i < n; i++) {
    if (n % i == 0) {
        flag = false;
        break;
    }
}
```

先假设该数为素数

退出循环



真题解析

思路分析

- ③ 统计正整数 a 到 b 范围内的素数个数（包含 a 和 b ）。

```
//遍历a~b之间所有的正整数
for (int n = a; n <= b; n++) {
    //判断当前正整数n是否为素数
    bool flag = true;
    for (int i = 2; i < n; i++) {
        if (n % i == 0) {
            flag = false;
            break;
        }
    }
```

根据flag的值累加
素数的个数

```
    if (flag) cnt++;
}
```

flag → **true** 素数
flag → **false** 非素数

- ④ 输出素数的个数。

```
cout << cnt << endl;
```



真题解析

【完整代码】

```
#include <iostream>
using namespace std;
int main() {
    int a = 0, b = 0, cnt = 0;
    cin >> a >> b;
    for (int n = a; n <= b; n++) {
        bool flag = true;
        for (int i = 2; i < n; i++) {
            if (n % i == 0) {
                flag = false;
                break;
            }
        }
        if (flag) cnt++;
    }
    cout << cnt << endl;
    return 0;
}
```



真题解析

(2023年3月) 自幂数判断

【问题描述】

自幂数是指，一个N位数，满足各位数字N次方之和是本身。

例如：153 是 3 位数，其每位数的 3 次方之和， $1^3 + 5^3 + 3^3 = 153$ ，因此 153 是自幂数；

1634 是 4 位数，其每位数的 4 次方之和， $1^4 + 6^4 + 3^4 + 4^4 = 1634$ ，因此 1634 是自幂数。

现在，输入若干个正整数，请判断它们是否是自幂数。

【输入描述】

输入第一行是一个正整数M，表示有M个待判断的正整数。约定 $1 \leq M \leq 100$ 。

从第 2 行开始的M行，每行一个待判断的正整数。约定这些正整数均小于 10^8 。

【输出描述】

输出M行，如果对应的待判断正整数为自幂数，则输出英文大写字母'T'， 否则输出英文大写字母'F'。

【提示】

不需要等到所有输入结束在依次输出，可以输入一个数就判断一个数并输出，再输入下一个数。



真题解析

(2023年3月) 自幂数判断

【输入样例1】

3

152

111

153

【输出样例1】

F

F

T

【输入样例2】

5

8208

548834

88593477

12345

5432

【输出样例2】

T

T

T

F

F

$$152 \rightarrow 1^3 + 5^3 + 2^3 = 134 \quad \times$$

$$111 \rightarrow 1^3 + 1^3 + 1^3 = 1 \quad \times$$

$$153 \rightarrow 1^3 + 5^3 + 3^3 = 153 \quad \checkmark$$

$$8208 \rightarrow 8^4 + 2^4 + 0^4 + 8^4 = 8208 \quad \checkmark$$

$$548834 \rightarrow 5^6 + 4^6 + 8^6 + 8^6 + 3^6 + 4^6 = 548834 \quad \checkmark$$

$$88593477 \rightarrow 8^8 + 8^8 + 5^8 + 9^8 + 3^8 + 4^8 + 7^8 + 7^8 = 88593477 \quad \checkmark$$

$$12345 \rightarrow 1^5 + 2^5 + 3^5 + 4^5 + 5^5 = 4425 \quad \times$$

$$5432 \rightarrow 5^4 + 4^4 + 3^4 + 2^4 = 978 \quad \times$$



真题解析

思路分析

- ① 输入待判断整数的个数m。

```
int m = 0;  
cin >> m;
```

- ② 输入m个数字，判断每个数字是否为自幂数。

```
int n = 0;  
for (int i = 1; i <= m; i++) {  
    cin >> n;
```

1) 计算当前正整数n是几位数

2) 判断当前正整数n是否为自幂数

```
}
```



真题解析

思路分析

1) 计算当前正整数 n 是几位数。

不改变 n 的值

记录位数

```
int t = n,  
int l = 0;  
while (t != 0) {  
    t /= 10;  
    l++;  
}
```

2) 判断当前正整数 n 是否为自幂数。 $153 \rightarrow 1^3 + 5^3 + 3^3 = 153$ ✓

求自幂数的方式：

a: 累加 n 的每位数的 l 次方

b: 判断累加和是否与 n 相等



真题解析

思路分析

2) 判断当前正整数 n 是否为自幂数

求自幂数的方式:

a: 累加 n 的每位数的 l 次方

b: 判断累加和是否与 n 相等

自幂数 T

非自幂数 F

记录累加和

```
int sum = 0;
t = n; //t的值重置为n
while (t != 0){
    int d = t % 10; //计算出每个数位上数字
    t /= 10;
    sum += pow(d, l); //累加每位数的次方
}
if (sum == n)
    cout << "T" << endl;
else
    cout << "F" << endl;
```



真题解析

【完整代码】

```
#include <iostream>
#include <cmath>
using namespace std;
int main() {
    int m = 0;
    cin >> m;
    for (int i = 1; i <= m; i++){
        int n = 0;
        cin >> n;
        int t = n, l = 0;
        while (t != 0) {
            t /= 10;
            l++;
        }
    }
```

提供pow函数

```
int sum = 0;
t = n;
while (t != 0) {
    int d = t % 10;
    t /= 10;
    sum += pow(d, l);
}
if (sum == n)
    cout << "T" << endl;
else
    cout << "F" << endl;
}
return 0;
}
```



真题解析

(2023年3月) 百鸡问题🐔

【问题描述】

“百鸡问题”是出自我国古代《张丘建算经》的著名数学问题。大意为：“每只公鸡 5 元，每只母鸡 3 元，每 3 只小鸡 1 元；现在有 100 元，买了 100 只鸡，共有多少种方案？”

小明很喜欢这个故事，他决定对这个问题进行扩展，并使用编程解决：如果每只公鸡 x 元，每只母鸡 y 元，每 z 只小鸡 1 元；

现在有 n 元，买了 m 只鸡，共有多少种方案？

【输入描述】

输入一行，包含五个整数，分别为问题描述中的 x 、 y 、 z 、 n 、 m 。

约定 $1 \leq x, y, z \leq 10$, $1 \leq n, m \leq 1000$ 。

【输出描述】

输出一行，包含一个整数 C ，表示有 C 种方案。



真题解析

(2023年3月) 百鸡问题🐔

【输入样例1】

5 3 3 100 100

【输出样例1】

4

【样例1解释】

这就是问题描述中的“百鸡问题”。

4 种方案分别为：公鸡 0 只、母鸡 25 只、小鸡 75 只；

公鸡 4 只、母鸡 18 只、小鸡 78 只；

公鸡 8 只、母鸡 11 只、小鸡 81 只；

公鸡 12 只、母鸡 4 只、小鸡 84 只。

【输入样例2】

1 1 1 100 100

【输出样例2】

5151



真题解析

思路分析

① 列出题目给出信息。

- 鸡的价格：公鸡： x /只；母鸡： y /只；小鸡： z 只1元
- 手中钱数： n
- 购鸡总数： m

② 根据题目给出信息列出解题的条件。

- 公鸡数 $\times x$ + 母鸡数 $\times y$ + 小鸡数 $\div z = n$
- 公鸡数 + 母鸡数 + 小鸡数 = m

③ 使用枚举法列出所有公鸡、母鸡可能的数量。

公鸡最少0只，最多 n/x 只

0	1	2	3		n/x
---	---	---	---	-------	-------

母鸡最少0只，最多 n/y 只

0	1	2	3		n/y
---	---	---	---	-------	-------

利用条件求出小鸡数：(总钱数-买公鸡母鸡钱数) $\times z$



真题解析

思路分析

① 输入公鸡价格 x ，母鸡价格 y ， z 只小鸡一元，总钱数 n ，购买鸡数 m 。

```
int x, y, z, n, m;  
cin >> x >> y >> z >> n >> m;
```

② 枚举公鸡、母鸡数量，计算小鸡数量，找出符合要求的购鸡方案数量。

- 公鸡数 $\times x$ + 母鸡数 $\times y$ + 小鸡数 $\div z = n$
- 公鸡数 + 母鸡数 + 小鸡数 = m

记录方案数量

```
int cnt=0;  
for(int a=0; a<=n/x; a++){ ➡ 枚举公鸡数量 (0~n/x)  
    for(int b=0; b<=n/y; b++){ ➡ 枚举母鸡数量 (0~n/y)  
        int c=(n-a*x-b*y)*z; ➡ 计算小鸡数量  
        当满足如下条件时 购鸡方案数量+1:  
        条件: 公鸡数 + 母鸡数 + 小鸡数 = m  
    }  
}
```



真题解析

思路分析

② 枚举公鸡、母鸡数量，计算小鸡数量，找出符合要求的购鸡方案数量。

```
int cnt=0;
for(int a=0; a<=n/x; a++){
    for(int b=0; b<=n/y; b++){
        int c=(n-a*x-b*y)/z;
        if(a+b+c==m && a>=0 && b>=0 && c>=0) cnt++;
    }
}
```

例如：公鸡1元/只，母鸡1元/只，小鸡1只1元，总钱数3，购鸡总数为3。

公鸡数

母鸡数

0
1
2
3

0
1
2
3

公鸡数	母鸡数	小鸡数
0	0	3
1	1	1
2	2	-1
3	3	-3
...	...	...

保证公鸡、母鸡、小鸡的数量

大于等于0!!!



真题解析

【完整代码】

```
#include <iostream>
using namespace std;
int main() {
    int x, y, z, n, m;
    cin >> x >> y >> z >> n >> m;
    int cnt = 0;
    for (int a = 0; a <= n / x; a++) {
        for (int b = 0; b <= n / y; b++) {
            int c = (n - a * x - b * y) * z;
            if (a + b + c == m && a >= 0 && b >= 0 && c >= 0) cnt++;
        }
    }
    cout << cnt;
    return 0;
}
```